

第2章 Python可迭代对象

《Python程序设计》，董付国

第2章 Python可迭代对象

- Python中常用的容器对象有列表、元组、字符串、字典、集合，与range以及zip、filter等迭代器对象统称为可迭代对象。
- 从面向对象程序设计的角度来看，可迭代对象是指实现了特殊方法__iter__()的类的对象，迭代器对象是指同时实现了特殊方法__iter__()和__next__()的类的对象。

第2章 Python可迭代对象

	列表	元组	字典	集合
类型名称	<code>list</code>	<code>tuple</code>	<code>dict</code>	<code>set</code>
定界符	方括号[]	圆括号()	大括号{}	大括号{}
是否可变	是	否	是	是
是否有序	是	是	否	否
是否支持下标	是（使用序号作为下标）	是（使用序号作为下标）	是（使用“键”作为下标）	否
元素分隔符	逗号	逗号	逗号	逗号
对元素形式的要求	无	无	键:值	必须可哈希
对元素值的要求	无	无	“键”必须可哈希	必须可哈希
元素是否可重复	是	是	“键”不允许重复，“值”可以重复	否
元素查找速度	非常慢	很慢	非常快	非常快
新增和删除元素速度	尾部操作快 其他位置慢	不允许	快	快

2.1 列表

- 列表是Python中内置有序、可变序列，列表的所有元素放在一对中括号[]中，并使用逗号分隔开；
- 当列表元素增加或删除时，列表对象自动进行扩展或收缩内存，保证元素之间没有缝隙；
- 在Python中，一个列表中的数据类型可以各不相同，可以同时分别为整数、实数、字符串等基本类型，甚至是列表、元组、字典、集合以及其他自定义类型的对象。

```
[10, 20, 30, 40]  
['red flower', 'blue sky', 'green leaf']  
['Python', 3.14, 5, [10, 20]]  
[['file1', 200, 7], ['file2', 260, 9]]
```

2.1 列表

- 列表常用方法

方法	说明
<code>append(object, /)</code>	将任意对象 <code>object</code> 追加至当前列表的尾部
<code>clear()</code>	删除列表中的所有元素
<code>copy()</code>	返回当前列表对象的浅复制
<code>count(value, /)</code>	返回值为 <code>value</code> 的元素在当前列表中的出现次数
<code>extend(iterable, /)</code>	将有限长度的可迭代对象 <code>iterable</code> 中所有元素追加至当前列表的尾部
<code>insert(index,object, /)</code>	在当前列表的 <code>index</code> 位置前面插入对象 <code>object</code>
<code>index(value, start=0, stop=9223372036854775807, /)</code>	返回当前列表指定范围中第一个值为 <code>value</code> 的元素的索引，若不存在值为 <code>value</code> 的元素则抛出异常
<code>pop(index=-1, /)</code>	删除并返回当前列表中下标为 <code>index</code> 的元素
<code>remove(value, /)</code>	在当前列表中删除第一个值为 <code>value</code> 的元素
<code>reverse()</code>	对当前列表中的所有元素进行原地翻转，首尾交换
<code>sort(*, key=None, reverse=False)</code>	对当前列表中的元素进行原地排序

2.1.1 列表创建与删除

- 使用 “=” 直接将一个列表赋值给变量即可创建列表对象

```
>>> a_list = ['a', 'b', 'mpilgrim', 'z', 'example']  
>>> a_list = [] # 创建空列表
```

- 也可以使用 `list()` 函数将元组、`range` 对象、字符串或其他类型的可迭代对象类型的数据转换为列表。

```
>>> a_list = list((3,5,7,9,11))  
>>> a_list  
[3, 5, 7, 9, 11]  
>>> list(range(1,10,2))  
[1, 3, 5, 7, 9]  
>>> list('hello world')  
['h', 'e', 'l', 'l', 'o', ' ', 'w', 'o', 'r', 'l', 'd']  
>>> x = list() # 创建空列表
```

2.1.1 列表创建与删除

- 当不再使用时，使用`del`命令删除整个列表。

```
>>> del a_list
```

```
>>> a_list
```

```
Traceback (most recent call last):
```

```
  File "<pyshell#6>", line 1, in <module>
```

```
    a_list
```

```
NameError: name 'a_list' is not defined
```

2.1.2 列表元素的增加

(1) 可以使用“+”运算符将元素添加到列表中。

```
>>> aList = [3,4,5]
>>> aList = aList + [7]
>>> aList
[3, 4, 5, 7]
```

严格意义上来讲，这并不是真的为列表添加元素，而是**创建了一个新列表**，并将原列表中的元素和新元素依次复制到新列表的内存空间。由于涉及大量元素的复制，**该操作速度较慢**，在涉及大量元素添加时不建议使用该方法。

2.1.2 列表元素的增加

(2) 使用列表对象的`append()`方法在当前列表尾部追加元素，原地修改列表，是真正意义上的在列表尾部添加元素，速度较快。

```
>>> aList.append(9)
>>> aList
[3, 4, 5, 7, 9]
```

2.1.2 列表元素的增加

(3) 使用列表对象的`extend()`方法可以将另一个可迭代对象的所有元素添加至该列表对象尾部。通过`extend()`方法来增加列表元素也不改变其内存首地址，属于原地操作。

```
>>> aList = [3, 4, 5, 7, 9]
>>> aList.extend([11,13])
>>> aList
[3, 4, 5, 7, 9, 11, 13]
>>> aList.extend((15,17))
>>> aList
[3, 4, 5, 7, 9, 11, 13, 15, 17]
```

2.1.2 列表元素的增加

- 运算符+=类似于列表的extend()方法。

```
>>> x = []  
>>> x += '1234'  
>>> x  
['1', '2', '3', '4']  
>>> x += range(3)  
>>> x  
['1', '2', '3', '4', 0, 1, 2]  
>>> x += map(str, range(3))  
>>> x  
['1', '2', '3', '4', 0, 1, 2, '0', '1', '2']
```

2.1.2 列表元素的增加

（4）使用列表对象的`insert()`方法将元素添加至列表的**指定位置**。

```
>>> aList = [3, 4, 5, 7, 9, 11, 13, 15, 17]
>>> aList.insert(3, 6)           # 在下标为3的位置插入元素6
>>> aList
[3, 4, 5, 6, 7, 9, 11, 13, 15, 17]
```

2.1.2 列表元素的增加

- 应尽量从列表尾部进行元素的增加与删除操作。
- ✓ 列表的`insert()`可以在列表的任意位置插入元素，但由于列表的自动内存管理功能，`insert()`方法会引起插入位置之后所有元素的移动，这会影响处理速度。
- ✓ 类似的还有后面介绍的`remove()`方法以及使用`pop()`函数弹出列表非尾部元素和使用`del`命令删除列表非尾部元素的情况。

2.1.2 列表元素的增加

（5）使用乘法来扩展列表对象，将列表与整数相乘，生成一个新列表，新列表是原列表中元素的重复。

```
>>> aList = [3, 5, 7]
>>> aList * 3
[3, 5, 7, 3, 5, 7, 3, 5, 7]
```

2.1.2 列表元素的增加

- 当使用*运算符将包含列表的列表重复并创建新列表时，并不是复制子列表值，而是复制已有元素的引用。因此，当修改其中一个值时，相应的引用也会被修改。

```
>>> x = [[None] * 2] * 3
>>> x
[[None, None], [None, None], [None, None]]
>>> x[0][0] = 5
>>> x
[[5, None], [5, None], [5, None]]
```

2.1.3 列表元素的删除

(1) 使用del命令删除列表中的指定位置上的元素。

```
>>> a_list = [3, 5, 7, 9, 11]
>>> del a_list[1]
>>> a_list
[3, 7, 9, 11]
```


2.1.3 列表元素的删除

(2) 使用列表的`pop()`方法删除并返回*指定位置*（默认为最后一个）上的元素，如果给定的索引超出了列表的范围则抛出异常。

```
>>> a_list = list((3,5,7,9,11))
>>> a_list.pop()
11
>>> a_list
[3, 5, 7, 9]
>>> a_list.pop(1)
5
>>> a_list
[3, 7, 9]
```

2.1.3 列表元素的删除

(3) 使用列表对象的`remove()`方法删除首次出现的指定元素，如果列表中不存在要删除的元素，则抛出异常。

```
>>> a_list = [3, 5, 7, 9, 7, 11]
>>> a_list.remove(7)
>>> a_list
[3, 5, 9, 7, 11]
```

2.1.3 列表元素的删除

- 代码编写好后必须要经过反复测试，不能满足于几次测试结果正确。例如，下面的代码成功地删除了列表中的重复元素，执行结果是完全正确的。

```
>>> x = [1, 2, 1, 2, 1, 2, 1, 2, 1]
>>> for i in x:
    if i == 1:
        x.remove(i)

>>> x
[2, 2, 2, 2]
```

2.1.3 列表元素的删除

- 然而，上面这段代码的逻辑是错误的。同样的代码，仅仅是所处理的数据发生了一点变化，然而当循环结束后却发现并没有把所有的“1”都删除，只是删除了一部分。

```
>>> x = [1, 2, 1, 2, 1, 1, 1]
>>> for i in x:
    if i == 1:
        x.remove(i)

>>> x
[2, 2, 1]
```

2.1.3 列表元素的删除

- 两组数据的本质区别在于，第一组数据中没有连续的“1”，而第二组数据中存在连续的“1”。出现这个问题的原因是列表的自动内存管理功能。
- 在删除列表元素时，Python会自动对列表内存进行收缩并移动列表元素以保证所有元素之间没有空隙，增加列表元素时也会自动扩展内存并对元素进行移动以保证元素之间没有空隙。每当插入或删除一个元素之后，该元素位置后面所有元素的索引就都改变了。

2.1.3 列表元素的删除

- 正确的代码:

```
>>> x = [1, 2, 1, 2, 1, 1, 1]
```

```
>>> for i in range(len(x)-1, -1, -1):
```

```
    if x[i] == 1:
```

```
        del x[i]
```

从后往前删

2.1.4 列表元素访问与计数

- 使用下标直接访问列表元素，如果指定下标不存在，则抛出异常。

```
>>> aList[3]
```

```
6
```

```
>>> aList[3] = 5.5
```

```
>>> aList
```

```
[3, 4, 5, 5.5, 7, 9, 11, 13, 15, 17]
```

```
>>> aList[15]
```

```
Traceback (most recent call last):
```

```
  File "<pyshell#34>", line 1, in <module>
```

```
    aList[15]
```

```
IndexError: list index out of range
```

2.1.4 列表元素访问与计数

- 使用列表对象的`index()`方法获取指定元素首次出现的下标，若列表对象中不存在指定元素，则抛出异常。

```
>>> aList = [3, 4, 5, 5.5, 7, 9, 11, 13, 15, 17]
```

```
>>> aList.index(7)
```

```
4
```

```
>>> aList.index(100)
```

```
Traceback (most recent call last):
```

```
  File "<pyshell#36>", line 1, in <module>
```

```
    aList.index(100)
```

```
ValueError: 100 is not in list
```


2.1.4 列表元素访问与计数

- 使用列表对象的count()方法统计指定元素在列表对象中出现的次数。

```
>>> aList = [3, 4, 5, 5.5, 7, 9, 11, 13, 15, 17]
```

```
>>> aList.count(7)
```

```
1
```

```
>>> aList.count(0)
```

```
0
```

```
>>> aList.count(8)
```

```
0
```

2.1.5 元素存在性判断

- 使用in关键字来判断一个值是否存在于列表中，返回结果为“True”或“False”。

```
>>> aList = [3, 4, 5, 5.5, 7, 9, 11, 13, 15, 17]
```

```
>>> 3 in aList
```

```
True
```

```
>>> 18 in aList
```

```
False
```

```
>>> bList = [[1], [2], [3]]
```

```
>>> 3 in bList
```

```
False
```

```
>>> 3 not in bList
```

```
True
```

```
>>> [3] in bList
```

```
True
```

```
>>> aList = [3, 5, 7, 9, 11]
```

```
>>> bList = ['a', 'b', 'c', 'd']
```

```
>>> (3, 'a') in zip(aList, bList)
```

```
True
```

```
>>> for a, b in zip(aList, bList):  
    print(a, b)
```

2.1.6 切片操作

- 切片适用于列表、元组、字符串、**range**对象等类型，但作用于列表时功能最强大。可以使用切片来**截取**列表中的任何部分，得到一个新列表，也可以通过切片来**修改**和**删除**列表中部分元素，甚至可以通过切片操作为列表对象**增加**元素。
- 切片使用**2个冒号分隔的3个数字**来完成，例如`data[start:stop:step]`:
 - ✓ **第一个数字start**表示切片开始位置（默认为0）。
 - ✓ **第二个数字stop**表示切片截止（但不包含）位置（默认为列表长度）。
 - ✓ **第三个数字step**表示切片的步长（默认为1），当步长省略时可以顺便省略最后一个冒号。
- 切片操作不会因为下标越界而抛出异常，而是简单地在列表尾部截断或者返回一个空列表，代码具有**更强的健壮性**。

2.1.6 切片操作

```
>>> alist = [3, 4, 5, 6, 7, 9, 11, 13, 15, 17]
>>> alist[:]          # 返回包含所有元素的新列表
[3, 4, 5, 6, 7, 9, 11, 13, 15, 17]
>>> alist[::-1]       # 逆序的所有元素
[17, 15, 13, 11, 9, 7, 6, 5, 4, 3]
>>> alist[::2]        # 偶数位置，隔一个取一个
[3, 5, 7, 11, 15]
>>> alist[1::2]       # 奇数位置，隔一个取一个
[4, 6, 9, 13, 17]
>>> alist[3::]        # 从下标3开始的所有元素
[6, 7, 9, 11, 13, 15, 17]
>>> alist[3:6]        # 下标在[3, 6)之间的所有元素
[6, 7, 9]
>>> alist[-3:]        # 最后3个元素
[13, 15, 17]
>>> alist[:-6]        # 倒数第6个（不包含）之前的元素
[3, 4, 5, 6]
>>> alist[0:100:1]    # 前100个元素，自动截断
[3, 4, 5, 6, 7, 9, 11, 13, 15, 17]
>>> alist[100:]       # 下标100之后的所有元素，自动截断
[]
>>> alist[100]        # 直接使用下标访问会发生越界
IndexError: list index out of range
```

2.1.6 切片操作

■ 可以使用切片来**原地修改**列表内容

```
>>> aList = [3, 5, 7]
>>> aList[len(aList):] = [9]      # 在尾部追加元素
>>> aList
[3, 5, 7, 9]
>>> aList[:3] = [1, 2, 3]        # 替换前3个元素
>>> aList
[1, 2, 3, 9]
>>> aList[:3] = []              # 删除前3个元素
>>> aList
[9]
>>> aList = list(range(10))
>>> aList
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
>>> aList[::2] = [0]*5           # 替换偶数位置上的元素
>>> aList
[0, 1, 0, 3, 0, 5, 0, 7, 0, 9]
>>> aList[::2] = [0]*3           # 切片不连续，两侧元素个数必须一样多
ValueError: attempt to assign sequence of size 3 to extended slice of size 5
```

2.1.6 切片操作

■ 使用del与切片结合来删除列表元素

```
>>> aList = [3, 5, 7, 9, 11]
```

```
>>> del aList[:3]
```

```
>>> aList
```

```
[9, 11]
```

```
>>> aList = [3, 5, 7, 9, 11]
```

```
>>> del aList[::2]
```

```
>>> aList
```

```
[5, 9]
```

删除前3个元素

删除偶数位置上的元素

2.1.6 切片操作

- 切片返回的是浅复制。所谓浅复制，是指生成一个新的列表，并且把原列表中所选元素的引用都复制到新列表中。如果原列表中只包含整数、实数、复数等基本类型或元组、字符串这样的不可变类型的数据，一般是没有问题的。

```
>>> aList = [3, 5, 7]
>>> bList = aList[:]
>>> aList == bList
True
>>> aList is bList
False
>>> id(aList) == id(bList)
False
>>> bList[1] = 8
>>> bList
[3, 8, 7]
>>> aList
[3, 5, 7]
```

切片，浅复制
两个列表的元素完全一样
但不是同一个对象
内存地址不一样
修改其中一个不会影响另一个

2.1.6 切片操作

- 如果原列表中包含列表之类的可变数据类型，由于浅复制时只是把子列表的引用复制到新列表中，这样的话修改任何一个都会影响另外一个。

```
>>> aList = [3, [5], 7]      # 列表aList中包含可变的列表对象
>>> bList = aList[:]        # 切片
>>> bList[1].append(6)      # 调用子列表的append()方法，这个方法是原地操作的
>>> bList
[3, [5, 6], 7]
>>> aList                   # aList受到影响
[3, [5, 6], 7]
```


2.1.6 切片操作

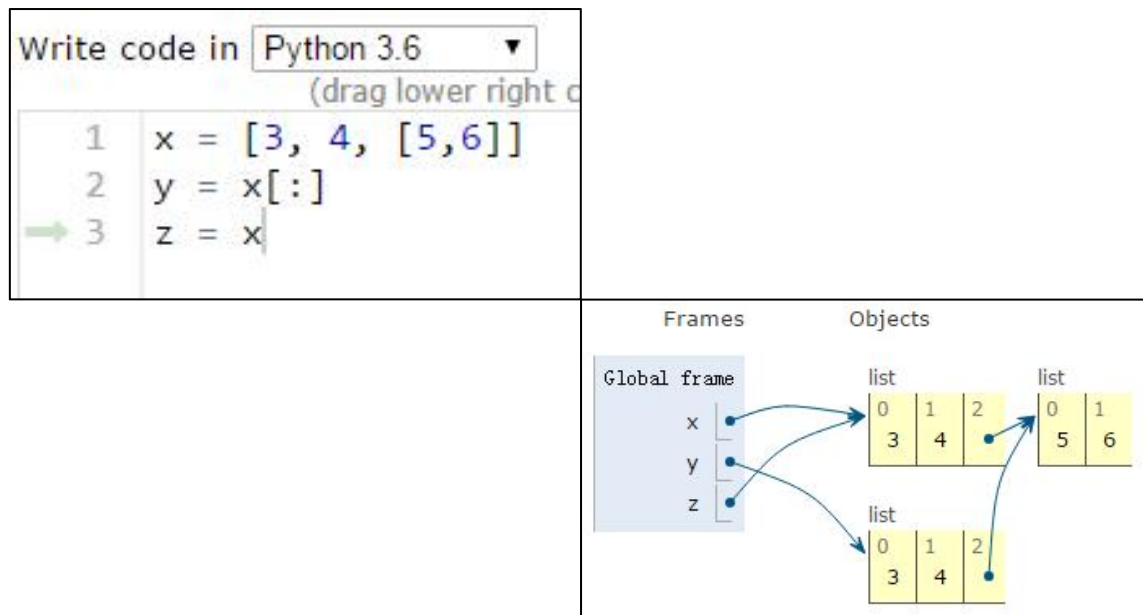
- 标准库copy中的deepcopy()函数实现深复制。所谓深复制，是指对原列表中的元素进行递归，把所有的值都复制到新列表中，对嵌套的子列表不再是复制引用。新列表和原列表是互相独立，修改任何一个都不会影响另外一个。

```
>>> aList = [3, [5], 7]
>>> import copy
>>> bList = copy.deepcopy(aList) # 深赋值，递归复制，直到遇到可哈希对象
                                   # aList和bList完全独立，互相不影响

>>> aList == bList
True
>>> aList is bList
False
>>> bList[1].append(6)           # 修改bList不会影响aList
>>> bList
[3, [5, 6], 7]
>>> aList
[3, [5], 7]
```

2.1.6 切片操作

一个非常棒的网站: <http://pythontutor.com/live.html#mode=edit>



2.1.7 列表排序

- 使用列表对象的`sort()`方法进行原地排序，支持多种不同的排序方法。

```
>>> aList = [3, 4, 5, 6, 7, 9, 11, 13, 15, 17]
>>> import random
>>> random.shuffle(aList)
>>> aList
[3, 4, 15, 11, 9, 17, 13, 6, 7, 5]
>>> aList.sort() # 默认是升序排序，没有返回值
>>> aList
[3, 4, 5, 6, 7, 9, 11, 13, 15, 17]
>>> aList.sort(reverse=True) # 降序排序
>>> aList
[17, 15, 13, 11, 9, 7, 6, 5, 4, 3]
>>> aList.sort(key=lambda x:len(str(x))) # 按转换成字符串的长度排序
>>> aList
[9, 7, 6, 5, 4, 3, 17, 15, 13, 11]
>>> aList.sort(key=lambda num:sum(map(int,str(num))))
# 按各位数字之和升序排列，稳定排序
# 各位数字之和相等的保持原来的相对顺序

>>> aList
[11, 3, 4, 13, 5, 6, 15, 7, 17, 9]
```

2.1.7 列表排序

- 使用内置函数`sorted()`对列表进行排序并返回新列表。

```
>>> aList
```

```
[9, 7, 6, 5, 4, 3, 17, 15, 13, 11]
```

```
>>> sorted(aList)
```

```
[3, 4, 5, 6, 7, 9, 11, 13, 15, 17]
```

```
>>> sorted(aList, reverse=True)
```

```
[17, 15, 13, 11, 9, 7, 6, 5, 4, 3]
```

升序排序

降序排序

2.1.8 用于序列操作的常用内置函数

- `all()`和`any()`: `all()`函数用来测试列表、元组等序列对象以及`map`对象、`zip`对象等类似对象中是否所有元素都等价于`True`, `any()`函数用来测试序列或可迭代对象中是否存在等价于`True`的元素。例如:

```
>>> all([1,2,3])
```

```
True
```

```
>>> all([0,1,2,3])
```

```
False
```

```
>>> any([0,1,2,3])
```

```
True
```

```
>>> any([0])
```

```
False
```

2.1.8 用于序列操作的常用内置函数

- `len()`: 返回列表中的元素个数，同样适用于元组、字典、集合、字符串等。
- `max()`、`min()`: 返回列表中的最大或最小元素，同样适用于元组、字典、集合、`range`对象等。
- `sum()`: 对列表的元素进行求和运算，对非数值型列表运算需要指定`start`参数，同样适用于元组、`range`。

```
>>> sum(range(1, 11))          # sum()函数的start参数默认为0
```

```
55
```

```
>>> sum(range(1, 11), 5)      # 指定start参数为5，等价于5+sum(range(1,11))
```

```
60
```

```
>>> sum([[1, 2], [3], [4]], []) # 这个操作占用空间较大，慎用
```

```
[1, 2, 3, 4]
```

2.1.8 用于序列操作的常用内置函数

■ zip()函数返回可迭代的zip对象。

```
>>> aList = [1, 2, 3]
```

```
>>> bList = [4, 5, 6]
```

```
>>> cList = zip(aList, bList)
```

返回zip对象

```
>>> cList
```

```
<zip object at 0x000000003728908>
```

```
>>> list(cList)
```

把zip对象转换成列表

```
[(1, 4), (2, 5), (3, 6)]
```

2.1.8 用于序列操作的常用内置函数

- `enumerate()`: 枚举列表元素，返回枚举对象，其中每个元素为包含下标和值的元组。该函数对元组、字符串同样有效。

```
>>> for index, num in enumerate([1,2,3,4]): # 序列解包, 见2.2.3节
    print((index,num), end=',')
(0, 1),(1, 2),(2, 3),(3, 4),
>>> for index, (num,ch) in enumerate(zip([1,2,3], 'ab')):
    print(index, num, ch)
0 1 a
1 2 b
```


2.1.9 列表推导式

- 列表推导式（也称列表解析式）使用非常简洁的方式来快速生成满足特定需求的列表，代码具有非常强的可读性。

- 列表推导式语法形式为：

```
[expression  for element1 in iterable1 if condition1  
              for element2 in iterable2 if condition2  
              for element3 in iterable3 if condition3  
              ...  
              for elementN in iterableN if conditionN]
```

2.1.9 列表推导式

- 阿凡提与国王比赛下棋，国王说要是自己输了的话阿凡提想要什么他都可以拿出来。阿凡提说那就要点米吧，棋盘一共**64**个小格子，在第一个格子里放**1**粒米，第二个格子里放**2**粒米，第三个格子里放**4**粒米，第四个格子里放**8**粒米，以此类推，后面每个格子里的米都是前一个格子里的**2**倍，一直把**64**个格子都放满。需要多少粒米呢？

```
>>> sum([2**i for i in range(64)])
```

```
18446744073709551615
```

```
>>> int('1'*64, 2) #第二个参数 2 表示将前面的字符串按照二进制来进行转换
```

```
18446744073709551615
```

2.1.9 列表推导式

■ 列表推导式在内部实际上是一个循环结构，只是形式更加简洁，例如：

```
>>> aList = [x*x for x in range(10)]
```

相当于

```
>>> aList = []  
>>> for x in range(10):  
    aList.append(x*x)
```

也相当于

```
>>> aList = list(map(lambda x: x*x, range(10)))
```

2.1.9 列表推导式

- 使用列表推导式实现嵌套列表的平铺

```
>>> vec = [[1,2,3], [4,5,6], [7,8,9]]
>>> [num for elem in vec for num in elem]
[1, 2, 3, 4, 5, 6, 7, 8, 9]
```

相当于

```
>>> vec = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
>>> result = []
>>> for elem in vec:
    for num in elem:
        result.append(num)

>>> result
[1, 2, 3, 4, 5, 6, 7, 8, 9]
```

2.1.9 列表推导式

- 如果不确定嵌套层数，可以这样做：

```
data = [[[1],[2],[3]], [[4],[5],[6]], [[7],[8],[9]]]
print(data)
result = []
def take_out(arr):
    for item in arr:
        if isinstance(item, int):
            result.append(item)
        else:
            take_out(item)

take_out(data)
print(result)
```

2.1.9 列表推导式

- 过滤不符合条件的元素

```
>>> import os
>>> [filename for filename in os.listdir('.')
      if filename.endswith(('.py', '.pyw'))]    # 略去结果
>>> aList = [-1, -4, 6, 7.5, -2.3, 9, -11]
>>> [i for i in aList if i>0]
[6, 7.5, 9]
```

2.1.9 列表推导式

- 在列表推导式中使用多个循环，实现多序列元素的任意组合

```
>>> [(x, y) for x in range(3) for y in range(3)]  
[(0, 0), (0, 1), (0, 2), (1, 0), (1, 1), (1, 2), (2, 0), (2, 1), (2, 2)]  
>>> from itertools import product  
>>> list(product(range(3), range(3)))      # 等价代码  
[(0, 0), (0, 1), (0, 2), (1, 0), (1, 1), (1, 2), (2, 0), (2, 1), (2, 2)]  
>>> [(x, y) for x in [1, 2, 3] for y in [3, 1, 4] if x!=y]  
[(1, 3), (1, 4), (2, 3), (2, 1), (2, 4), (3, 1), (3, 4)]
```

2.1.9 列表推导式

■ 列表推导式中使用函数或复杂表达式

```
>>> def f(v):  
    if v%2 == 0:  
        v = v**2  
    else:  
        v = v+1  
    return v
```

```
>>> [f(v) for v in [2, 3, 4, -1] if v>0]
```

```
[4, 4, 16]
```

```
>>> [v**2 if v%2 == 0 else v+1 for v in [2, 3, 4, -1] if v>0]
```

```
[4, 4, 16]
```

```
>>> x = list(range(10))
```

```
>>> [item>5 for item in x]
```

```
[False, False, False, False, False, False, True, True, True, True]
```


补充：计算样本标准差

```
x = [3, 7, 23, 21, 8, 10]
avg = sum(x) / len(x)
s = [(xi-avg)**2 for xi in x]
s = (sum(s)/len(s)) ** 0.5
print(s)
```

2.2 元组

- 元组和列表类似，但属于**不可变序列**，元组一旦创建，用任何方法都不可以修改其元素。
- 元组的定义方式和列表相同，但定义时所有元素放在一对圆括号“（）”中，而不是方括号中。

2.2.1 元组创建与删除

- 使用 “=” 将一个元组赋值给变量

```
>>> a_tuple = ('a', 'b', 'mpilgrim', 'z', 'example')
```

```
>>> a_tuple
```

```
('a', 'b', 'mpilgrim', 'z', 'example')
```

```
>>> a = (3)
```

```
>>> a
```

```
3
```

```
>>> a = (3,)
```

包含一个元素的元组，最后必须多写个逗号

```
>>> a
```

```
(3,)
```

```
>>> a = 3,
```

也可以这样创建元组

```
>>> a
```

```
(3,)
```

```
>>> x = ()
```

空元组

2.2.1 元组创建与删除

- 使用tuple函数将其他序列转换为元组

```
>>> tuple('abcdefg')
```

```
('a', 'b', 'c', 'd', 'e', 'f', 'g')
```

```
>>> aList
```

```
[-1, -4, 6, 7.5, -2.3, 9, -11]
```

```
>>> tuple(aList)
```

```
(-1, -4, 6, 7.5, -2.3, 9, -11)
```

```
>>> s = tuple()
```

```
>>> s
```

```
()
```

- 使用del可以删除元组对象，不能删除元组中的元素

把字符串转换为元组

把列表转换为元组

空元组

2.2.2 元组与列表的区别

- ❖ 元组一旦定义就**不允许更改**。
- ❖ 元组没有`append()`、`extend()`和`insert()`等方法，**无法向元组中添加元素**。
- ❖ 元组没有`remove()`或`pop()`方法，也无法对元组元素进行`del`操作，**不能从元组中删除元素**。

2.2.2 元组与列表的区别

- ❖ 元组的速度比列表略快，占用空间略少。如果定义了一系列常量值，而所需做的仅是对它进行遍历，那么一般使用元组而不用列表。
- ❖ 元组对不需要改变的数据进行“写保护”将使得代码更加安全。
- ❖ 元组可用作字典的“键”，也可以作为集合的元素。列表不能作为字典的“键”，包含列表、字典、集合或其他类型可变对象的元组也不能做字典的“键”。

2.2.3 序列解包

- 可以使用序列解包功能对多个变量同时赋值，要求等号两侧的长度相同。

```
>>> x, y, z = 1, 2, 3                # 多个变量同时赋值
>>> v_tuple = (False, 3.5, 'exp')
>>> (x, y, z) = v_tuple
>>> x, y, z = v_tuple
>>> x, y, z = range(3)               # 可以对range对象进行序列解包
>>> x, y, z = iter([1, 2, 3])        # 使用迭代器对象进行序列解包
>>> x, y, z = map(str, range(3))     # 使用可迭代的map对象进行序列解包
>>> a, b = b, a                      # 交换两个变量的值
>>> x, y, z = sorted([1, 3, 2])      # sorted()函数返回排序后的列表
>>> a, b, c = 'ABC'                 # 字符串也支持序列解包
>>> x = [1, 2, 3, 4, 5, 6]
>>> x[:3] = map(str, range(5))       # 切片也支持序列解包
```

2.2.3 序列解包

- 序列解包对于列表和字典同样有效

```
>>> s = {'a':1, 'b':2, 'c':3}
```

```
>>> b, c, d = s.items()
```

```
>>> b                                     # Python 3.6之后的版本和Python 3.5之前的版本结果略有不同
```

```
('a', 1)
```

```
>>> b, c, d = s      # 使用字典时不用太多考虑元素的顺序
```

```
>>> b
```

```
'a'
```

```
>>> b, c, d = s.values()
```

```
>>> print(b, c, d)
```

```
1 2 3
```


2.2.3 序列解包

■ 序列解包遍历多个序列

```
>>> keys = ['a', 'b', 'c', 'd']  
>>> values = [1, 2, 3, 4]  
>>> for k, v in zip(keys, values):  
    print((k, v), end=' ')
```

```
('a', 1) ('b', 2) ('c', 3) ('d', 4)
```

2.2.4 生成器表达式

- 生成器表达式的结果是一个生成器对象。使用生成器对象的元素时，可以根据需要将其转化为列表或元组，也可以使用生成器对象__next__()方法或内置函数next()获取下一个元素，或者使用for循环遍历其中的元素。
- 生成器对象具有惰性求值的特点，只在需要时生成新元素，空间占用非常少，尤其适合大数据处理的场合。
- 不管用哪种方法访问生成器对象，都无法再次访问已访问过的元素。

2.2.4 生成器表达式

- 使用生成器对象 `__next__()` 方法或内置函数 `next()` 进行遍历

```
>>> g = ((i+2)**2 for i in range(10)) # 创建生成器对象
>>> g
<generator object <genexpr> at 0x0000000003095200>
>>> tuple(g) # 将生成器对象转换为元组
(4, 9, 16, 25, 36, 49, 64, 81, 100, 121)
>>> list(g) # 生成器对象已遍历结束，没有元素了
[]
>>> g = ((i+2)**2 for i in range(10)) # 重新创建生成器对象
>>> g.__next__() # 使用生成器对象的__next__()方法获取元素
4
>>> g.__next__() # 获取下一个元素
9
>>> next(g) # 使用函数next()获取生成器对象中的元素
16
```

2.2.4 生成器表达式

- 使用for循环直接迭代生成器对象中的元素

```
>>> g = ((i+2)**2 for i in range(10))
```

```
>>> for item in g:                                     # 使用循环直接遍历生成器对象中的元素
    print(item, end=' ')
```

```
4 9 16 25 36 49 64 81 100 121
```

2.3 字典

- 字典是可变容器对象。
- 定义字典时，每个元素的键和值用冒号分隔，元素之间用逗号分隔，所有的元素放在一对大括号“{ }”中。
- 字典中的键可以为任意不可变数据，比如整数、实数、复数、字符串、元组等等。
- `globals()`返回包含当前作用域内所有全局变量和值的字典。
- `locals()`返回包含当前作用域内所有局部变量和值的字典。

2.3.1 字典创建与删除

- 使用“=”将一个字典赋值给一个变量

```
>>> a_dict = {'server': 'db.diveintopython3.org', 'database': 'mysql'}
```

```
>>> x = {}                                # 空字典
```

2.3.1 字典创建与删除

- 使用dict类的构造方法利用已有数据创建字典

```
>>> keys = ['a', 'b', 'c', 'd']
>>> values = [1, 2, 3, 4]
>>> dictionary = dict(zip(keys, values))
>>> dictionary
{'a': 1, 'b': 2, 'c': 3, 'd': 4}
>>> x = dict() # 空字典
>>> x
{}
>>> d = dict(name='Dong', age=37)
>>> d
{'name': 'Dong', 'age': 37}
```

2.3.1 字典创建与删除

- 使用字典类dict的fromkeys()方法，以给定内容为键，创建特定值的字典

```
>>> adict = dict.fromkeys(['name', 'age', 'sex'])
>>> adict
{'name': None, 'age': None, 'sex': None}
>>> data = dict.fromkeys(['a', 'b', 'c'], [])
>>> data
{'a': [], 'b': [], 'c': []}
# 所有“值”引用同一个列表
>>> data['a'].append(97)
# 影响另外两个“值”
>>> data
{'a': [97], 'b': [97], 'c': [97]}
>>> data['b'] = 98
# 修改了“值”的引用，不影响另外两个
>>> data
{'a': [97], 'b': 98, 'c': [97]}
```

- 可以使用del删除整个字典

2.3.2 字典元素的访问

- 以键作为下标可以读取字典元素，若键不存在则抛出异常

```
>>> aDict = {'name':'Dong', 'sex':'male', 'age':37}
>>> aDict['name']
'Dong'
>>> aDict['tel']                                # 键不存在，抛出异常
Traceback (most recent call last):
  File "<pyshell#53>", line 1, in <module>
    aDict['tel']
KeyError: 'tel'
```

2.3.2 字典元素的访问

- 使用字典对象的`get()`方法获取指定键对应的值，并且可以在键不存在的时候返回指定值。

```
>>> print(aDict.get('address'))
```

```
None
```

```
>>> print(aDict.get('address', 'SDIBT'))
```

```
SDIBT
```

2.3.2 字典元素的访问

```
>>> aDict = {'name':'Dong', 'sex':'male', 'age':37}
>>> for key in aDict:                                # 不加特殊说明，默认输出“键”
    print(key)
```

name

sex

age

```
>>> aDict.keys()                                     # 返回所有“键”
```

```
dict_keys(['name', 'sex', 'age'])
```

```
>>> aDict.values()                                   # 返回所有“值”
```

```
dict_values(['Dong', 'male', 37])
```

2.3.2 字典元素的访问

```
>>> for item in aDict.items():  
    print(item)
```

输出字典中所有元素

```
('name', 'Dong')  
('sex', 'male')  
('age', 37)
```

```
>>> for key, value in aDict.items():  
    print(key, value)
```

序列解包用法

```
name Dong  
sex male  
age 37
```

2.3.3 字典元素的添加与修改

- 当以指定键为下标为字典赋值时：1) 若键存在，则可以修改该键的值；2) 若不存在，则表示添加一个键、值对。

```
>>> aDict = {'age': 37, 'name': 'Dong', 'sex': 'male'}
>>> aDict['age'] = 38                      # 修改元素值
>>> aDict
{'age': 38, 'name': 'Dong', 'sex': 'male'}
>>> aDict['address'] = 'Yantai'           # 增加新元素
>>> aDict
{'age': 38, 'name': 'Dong', 'sex': 'male', 'address': 'Yantai'}
```

2.3.3 字典元素的添加与修改

- 使用`del`删除字典中指定键的元素
- 使用字典对象的`clear()`方法来删除字典中所有元素
- 使用字典对象的`pop()`方法删除并返回指定键的元素
- 使用字典对象的`popitem()`方法删除并返回字典中的一个元素

2.3.6 字典推导式

```
>>> {i:str(i) for i in range(1, 5)}  
{1: '1', 2: '2', 3: '3', 4: '4'}  
>>> x = ['A', 'B', 'C', 'D']  
>>> y = ['a', 'b', 'b', 'd']  
>>> {i:j for i,j in zip(x,y)}  
{'A': 'a', 'B': 'b', 'C': 'b', 'D': 'd'}
```

2.4 集合

- 集合是**无序、可变**容器对象，使用一对大括号界定，**元素不可重复**，同一个集合中每个元素都是唯一的。
- 集合中**只能包含数字、字符串、元组等不可变类型**（或者说可哈希）的数据，而不能包含列表、字典、集合等可变类型的数据。

2.4.1 集合的创建与删除

- 直接将集合赋值给变量

```
>>> a = {3, 5}
```

2.4.1 集合的创建与删除

- 使用**set**将其他类型数据转换为集合

```
>>> a_set = set(range(8,14))
```

```
>>> a_set
```

```
{8, 9, 10, 11, 12, 13}
```

```
>>> b_set = set([0, 1, 2, 3, 0, 1, 2, 3, 7, 8])    # 自动去除重复
```

```
>>> b_set
```

```
{0, 1, 2, 3, 7, 8}
```

```
>>> c_set = set()                                # 空集合
```

```
>>> c_set
```

```
set()
```

2.4.2 集合运算

- 交集、并集、差集、对称差集等运算

```
>>> a_set = set([8, 9, 10, 11, 12, 13])
```

```
>>> b_set = {0, 1, 2, 3, 7, 8}
```

```
>>> a_set | b_set
```

```
{0, 1, 2, 3, 7, 8, 9, 10, 11, 12, 13}
```

并集

```
>>> a_set & b_set
```

```
{8}
```

交集

```
>>> a_set - b_set
```

```
{9, 10, 11, 12, 13}
```

```
>>> a_set ^ b_set
```

```
{0, 1, 2, 3, 7, 9, 10, 11, 12, 13}
```

对称差集

2.4.2 集合运算

■ 集合包含关系测试

```
>>> x = {1, 2, 3}
```

```
>>> y = {1, 2, 5}
```

```
>>> z = {1, 2, 3, 4}
```

```
>>> x < y
```

比较集合大小/包含关系

```
False
```

```
>>> x < z
```

真子集

```
True
```

```
>>> y < z
```

```
False
```

```
>>> {1, 2, 3} <= {1, 2, 3}
```

子集

```
True
```

```
>>> max({1}, {2}, {3}, {4})
```

先假设第一个最大，然后检查有没有更大的

如果没有更大的集合，返回第一个

```
{1}
```

```
>>> min({1}, {2}, {3}, {4})
```

```
{1}
```

2.4.3 集合运用案例

- **例2-1** 生成不重复随机数的效率比较。

```
import random
import time
```

```
def RandomNumbers(number, start, end):
    '''使用列表来生成number个介于start和end之间的不重复随机数'''
    data = []
    n = 0
    while True:
        element = random.randint(start, end)
        if element not in data:
            data.append(element)
            n += 1
            if n == number:
                break
    return data
```

2.4.3 集合运用案例

```
def RandomNumbers2(number, start, end):  
    '''使用集合来生成number个介于start和end之间的不重复随机数'''  
    data = set()  
    while True:  
        data.add(random.randint(start, end))  
        if len(data) == number:  
            break  
    return data
```

2.4.3 集合运用案例

■ 例2-3 过滤无效书评。

- 很多人喜欢爬取书评，然后选择自己喜欢的书或者其他读者评价较高的书，这是一个非常好的思路，也是非常明智的做法。
- 然而，并不是每个消费者都会认真留言评论，也有部分消费者可能会复制了几个简单的句子或词作为评论。在爬取到原始书评之后可能需要进行简单的处理和过滤，这时就需要制定一个过滤的标准进行预处理，这也是数据处理与分析的关键内容之一。
- 在下面的代码中，采用了一个最简单的规则：正常书评中，重复的字应该不会超过一定的比例。

2.4.3 集合运用案例

```
comments = ['这是一本非常好的书，作者用心了',  
            '作者大大辛苦了',  
            '好书，感谢作者提供了这么多的好案例',  
            '书在运输的路上破损了，我好悲伤。。。',  
            '啊啊啊啊啊啊，我怎么才发现这么好的书啊，相见恨晚',  
            '好好好好好好好好好好',  
            '好难啊看不懂好难啊看不懂好难啊看不懂',  
            '书的内容很充实',  
            '你的书上好多代码啊，不过想想也是，编程的书嘛，肯定代码多一些',  
            '书很不错!!一级棒!!买书就上当当，正版，价格又实惠，让人放心!!! ',  
            '无意中来到你小铺就淘到心仪的宝贝，心情不错!',  
            '送给朋友的、很不错',  
            '这是一本好书，讲解内容深入浅出又清晰明了，推荐给所有喜欢阅读的朋友同好们。']  
result = filter(lambda s:len(set(s))/len(s)>0.5, comments)  
print('原始书评: ', *comments, sep='\n')  
print('='*30)  
print('过滤后的书评: ', *result, sep='\n')
```


2.4.3 集合运用案例

原始书评:

这是一本非常好的书，作者用心了

作者大大辛苦了

好书，感谢作者提供了这么多的好案例

书在运输的路上破损了，我好悲伤。。。

啊啊啊啊啊啊，我怎么才发现这么好的书啊，相见恨晚

好好好好好好好好好好

好难啊看不懂好难啊看不懂好难啊看不懂

书的内容很充实

你的书上好多代码啊，不过想想也是，编程的书嘛，肯定代码多一些

书很不错!!一级棒!!买书就上当当，正版，价格又实惠，让人放心!!!

无意中来到你小铺就淘到心仪的宝贝，心情不错!

送给朋友的、很不错

这是一本好书，讲解内容深入浅出又清晰明了，推荐给所有喜欢阅读的朋友同好们。

=====

过滤后的书评:

这是一本非常好的书，作者用心了

作者大大辛苦了

好书，感谢作者提供了这么多的好案例

书在运输的路上破损了，我好悲伤。。。

啊啊啊啊啊啊，我怎么才发现这么好的书啊，相见恨晚

书的内容很充实

你的书上好多代码啊，不过想想也是，编程的书嘛，肯定代码多一些

书很不错!!一级棒!!买书就上当当，正版，价格又实惠，让人放心!!!

无意中来到你小铺就淘到心仪的宝贝，心情不错!

送给朋友的、很不错

这是一本好书，讲解内容深入浅出又清晰明了，推荐给所有喜欢阅读的朋友同好们。

2.5 再谈内置方法sorted()

- 列表对象提供了`sort()`方法支持原地排序，而内置函数`sorted()`返回新列表，并不对原列表进行任何修改。
- `sorted()`方法可以对列表、元组、字典、`range`对象等进行排序。
- 列表的`sort()`方法和内置函数`sorted()`都支持`key`参数实现复杂排序要求。

2.5 再谈内置方法sorted()

```
>>> gameresult = [{ 'name': 'Bob', 'wins': 10, 'losses': 3, 'rating': 75.0},  
                  { 'name': 'David', 'wins': 3, 'losses': 5, 'rating': 57.0},  
                  { 'name': 'Carol', 'wins': 4, 'losses': 5, 'rating': 57.0},  
                  { 'name': 'Patty', 'wins': 9, 'losses': 3, 'rating': 72.8}]  
>>> sorted(gameresult, key=itemgetter('wins', 'name'))  
# 按'wins'升序, 该值相同的按'name'升序排序  
[{'name': 'David', 'wins': 3, 'losses': 5, 'rating': 57.0}, {'name':  
'Carol', 'wins': 4, 'losses': 5, 'rating': 57.0}, {'name': 'Patty',  
'wins': 9, 'losses': 3, 'rating': 72.8}, {'name': 'Bob', 'wins': 10,  
'losses': 3, 'rating': 75.0}]
```

2.5 再谈内置方法sorted()

- 问题描述：有一个整数列表，要求调整元素顺序，把所有奇数都放到前面，偶数都放到后面。

```
>>> from random import randint
>>> x = [randint(1,100) for i in range(20)]
>>> x
[19, 32, 76, 82, 23, 63, 38, 50, 20, 30, 39, 14, 19, 50, 81, 27, 77, 12, 55, 29]
>>> sorted(x, key=lambda item:item%2==0)
[19, 23, 63, 39, 19, 81, 27, 77, 55, 29, 32, 76, 82, 38, 50, 20, 30, 14, 50, 12]
```

2.6 复杂数据结构

- 在解决实际问题时，还经常需要用到其他复杂的数据结构，如堆、栈、队列、树、图等等。
- 有些结构Python已经提供，而有些则需要自己利用基本数据结构来实现。

2.6.1 堆

```
>>> import heapq                                     # heapq和random是Python标准库
>>> import random
>>> data = list(range(10))
>>> data
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
>>> random.choice(data)                               # 随机选择一个元素
9
>>> random.shuffle(data)                             # 随机打乱顺序
>>> data
[6, 1, 3, 4, 9, 0, 5, 2, 8, 7]
>>> heap=[]
>>> for n in data:                                    # 建堆
    heapq.heappush(heap,n)

>>> heap
[0, 2, 1, 4, 7, 3, 5, 6, 8, 9]
>>> heapq.heappush(heap,0.5)                          # 入堆，自动重建
>>> heap
[0, 0.5, 1, 4, 2, 3, 5, 6, 8, 9, 7]
>>> heapq.heappop(heap)                              # 出堆，自动重建
0
```

2.6.1 堆

```
>>> myheap = [1,2,3,5,7,8,9,4,10,333]
>>> heapq.heapify(myheap)           # 建堆
>>> myheap
[1, 2, 3, 4, 7, 8, 9, 5, 10, 333]
>>> heapq.heapreplace(myheap,6)     # 弹出最小元素，同时插入新元素
1
>>> myheap
[2, 4, 3, 5, 7, 8, 9, 6, 10, 333]
>>> heapq.nlargest(3, myheap)        # 返回前3个最大的元素，第2个参数不必须是堆
[333, 10, 9]
>>> heapq.nsmallest(3, myheap)      # 返回前3个最小的元素
[2, 3, 4]
```

2.6.2 队列

- 可以使用列表来模拟队列结构

```
>>> x = [1, 2, 3, 4]
```

```
>>> x.pop(0)
```

```
1
```

```
>>> x.pop(0)
```

```
2
```

```
>>> x.append(5)
```

```
>>> x.pop(0)
```

```
3
```

```
>>> x.pop(0)
```

```
4
```

```
>>> x.pop(0)
```

```
5
```

```
>>> x.pop(0)
```

```
IndexError: pop from empty list
```


2.6.2 队列

```
>>> import queue          #queue是Python标准库
>>> q = queue.Queue()
>>> q.put(0)              # 入队
>>> q.put(1)
>>> q.put(2)
>>> q.queue
deque([0, 1, 2])
>>> q.get()              # 出队
0
>>> q.queue
deque([1, 2])
>>> q.get()
1
>>> q.queue
deque([2])
```


2.6.3 栈

- 栈是一种“后进先出（LIFO）”或“先进后出（FILO）”的数据结构。
- Python列表本身就可以实现栈结构的基本操作。例如，列表对象的`append()`方法是在列表尾部追加元素，类似于入栈操作；`pop()`方法默认是弹出并返回列表的最后一个元素，类似于出栈操作。
- 但是直接使用Python列表对象模拟栈操作并不是很方便，例如当列表为空时再执行`pop()`出栈操作时则会抛出一个不很友好的异常；另外，也无法限制栈的大小。

2.6.3 栈

- 可以直接使用列表来实现栈结构

```
>>> myStack = []
```

```
>>> myStack.append(3)
```

```
>>> myStack.append(5)
```

```
>>> myStack.append(7)
```

```
>>> myStack
```

```
[3, 5, 7]
```

```
>>> myStack.pop()
```

```
7
```

```
>>> myStack.pop()
```

```
5
```

```
>>> myStack.pop()
```

```
3
```

```
>>> myStack.pop()
```

```
出错
```

2.6.4 链表

- 可直接使用列表来实现：

```
>>> linkTable = []
>>> linkTable.append(3)
>>> linkTable.append(5)
>>> linkTable
[3, 5]
>>> linkTable.insert(1,4)
>>> linkTable
[3, 4, 5]
>>> linkTable.remove(linkTable[1])
>>> linkTable
[3, 5]
```